Practical Session 1: Scheme (Functional Programming and List processing)

1 Getting started

Many LISP dialects have been implemented throughout history. For the purpose of this course, we will use one of the best-known of these dialects: Scheme. Scheme is actually a language by itself, described in a standard called the *Revised* ^{n} *Report on the Algorithmic Language Scheme* (R^nRS), where n is the version number of this language standard. Although the document is revised periodically (with R^7RS having been published in 2013 and a large edition still underway), the most commonly implemented version of this standard is still R^5RS[1].

One general-purpose implementation of (this standard of) Scheme is available in Racket[2]. Racket is actually the name of another LISP/Scheme dialect itself, but the Racket programming environment supports the use of both the Racket and R^5RS Scheme dialects, as well as some other variants. The Racket environment provides both a command-line interpreter as well as a GUI called DrRacket. In the GUI, you can write Scheme scripts or programs in the top panel. The bottom panel is an interpreter in which you can write expressions directly, or see the result of running the script or program typed in the top panel.

Every source file in DrRacket should start with an expression of the form `#lang language` where `language` tells the interpreter which dialect of Scheme you want to use. As mentioned above, for our practical sessions we will mostly use `#lang r5rs`. If you prefer to use your own text editor rather than the one provided by DrRacket, you can run your Scheme programs by executing the command `plt-r5rs -r myscript.scm`.

By default, the Scheme REPL[3] doesn't support line-control, meaning you can't type the up arrow to get back to the last line, etc. There is a workaround, which is to install a small utility called `rlwrap`. On Mac OS, you can get `rlwrap` on Homebrew (https://brew.sh): `brew install rlwrap`. On Linux, most distributions include a `rlwrap` package, e.g.: `apt-get install rlwrap`.

2 The basics

Scheme uses prefix notation. This means that applying a function f to two arguments x and y is written as follows:

[1] `http://www.schemers.org/Documents/Standards/R5RS/`
[2] `https://racket-lang.org/`
[3] Read-Eval-Print Loop

```
(f x y)
```

For instance, to compute $2 * (14 + 17)$, you need to write:

```
(* 2 (+ 14 17))
```

This may require a bit of getting used to, but you'll soon get the hang of it, don't worry. After all, it's only syntax.

Exercise 1

Write and evaluate the following expression in Scheme. (Hint: the Scheme function to calculate the square root is **sqrt**.)

$$\frac{(134 + 121) * \frac{74}{\sqrt{8}}}{97.1 - \frac{exp(7)}{78*71}}$$

It should evaluate to ~ 68.85.

Even control-flow statements use this prefix notation. For instance, an `if` condition would be written as:

```
(if test-expr true-expr false-expr)

(if (positive? a) "a is positive" "a is negative")
```

Exercise 2

Try to re-write the following Python piece of code in Scheme:

```
from math import sqrt
a = -8
if a > 0 :
        b = 2 * sqrt(a)
else :
        b = 7 * sqrt(-a) / -a

print(b)
```

To get you started, you can start from the following code skeleton:

```
(define a -8)
(define b ... )
(display b)
```

This code should print ~ 2.47.

3 Lists

Scheme is a dialect of LISP, which stands for *List Processing*. Lists are at the core of all LISP language dialects. They are the main data structure and the language is designed to efficiently manipulate lists in many ways, and to use them to define more complex data structures.

The most primitive expression types are: numbers (`1`, `1.3`, `1/3`, `1e3`, ...), symbols (`'a`, `'mysymbol`, `'*`, ...) (look at section 4.1.2 here for details), characters (`#\a`, `#\Z`), booleans (`#t` and `#f` for true and false), strings (`"Hello World!"`), ...

Values can be combined together with the **cons** function to form a *pair*. This pair constructor function takes two arguments and returns a pair of those two items. After evaluation, the Scheme interpreter will write a pair `(cons 'a 'b)` as: `(a . b)`

Once a pair has been constructed, retrieving its first and second elements again can be done through the infamous `car` and `cdr` functions.[4] (As a mnemonic device to remember which one retrieves the first value and which one retrieves the second, just remember that the letter "a" precedes the letter "d" in the alphabet.)

```
(define mypair (cons 'a  'b))
(car mypair)    ; -> Returns the symbol 'a
(cdr mypair)    ; -> Returns the symbol 'b
```

Note that a pair is not necessarily a *list* :

```
(list? mypair)  ; -> Returns #f (false)
```

A *list* is a special kind of pair (created with **cons**), in which the second element (the cdr) is either another list or the empty list `'()`.[5] For instance, the code below defines a list with one element:

```
(define mylist (cons 'a  '()))
(list? mylist)  ; -> returns #t (true)
```

The empty list (sometimes called null) is denoted as `'()` and is also considered as a list:

```
(list? '())              ; -> returns #t (true)
(list? (cdr mylist))     ; -> returns #t (true)
(null? '())              ; -> returns #t (true)
(null? mylist)           ; -> returns #f (false)
(null? (cdr mylist))     ; -> returns #t (true)
```

In the definition above you can already see that a list has a recursive definition: a list is a pair where the second element is again a list.[6] The empty list `'()` gives us a base case from which to build lists. Scheme provides facilities to build lists straightaway more easily than using multiple **cons**. For example, each of the lines below define valid lists:

[4] The names of car and cdr are historical since Lisp was originally implemented on the IBM 704 computer in the late 1950s, where **car** was used to retrieve the **c**ontents **a**ddress of a **r**egister and **cdr** was used to retrieve the **c**ontents **d**ecrement **r**egister.

[5] This is very similar to how lists are represented in the Prolog language.

[6] Like in Prolog.

```
(cons 'a (cons 'b (cons 'c (cons 'd '()))))
(list 1 2 3 4 "lists can have heterogeneous elements")
'(22 21 23 #f 45 0 'dog  "A cat is fine too")
```

Exercise 3
Try to guess the value of the following expressions. Then verify your answer in Scheme, using `display` to print your results. It can also be helpful to draw the data structures in these expressions as cons-cells to better understand what is happening and why.
Look up the meaning of pair accessors shorthands like `cadr` here, section 6.3.2. You can remember them as composition of `car` and `cdr`.

```
(cons 1 (cons 2 (cons 3 (cons 4 '() ))))
(car (cons 1 (cons 2 '())))
(car (cdr (list 1 2 3 4)))

(cadr '(1 2 3))
(caddr '(1 2 3))
(cdddr '(1 2 3))

(list (list 1 2 3) 4 5 6)
(cons 'a 'b)
(list? '(a . b))
(cons (cons 'a 'b) (cons 1 '() ))
(list? (cons (cons 'a 'b) (cons 1 '() )))

(cadar '((1 4 8 7) (1 3 4 2 5) (0 32 (54 4 46))))
(caddr '((1 4 8 7) (1 3 4 2 5) (0 32 (54 4 46))))
(caaddr '((1 4 8 7) (1 3 4 2 5) (0 32 (54 4 46))))
```

What is the difference between these two objects?

```
(cons 'a (cons 'b 'c))
(cons 'a (cons 'b (cons 'c '() )))
```

4 Functions

Scheme adheres to the functional programming paradigm. In the next session, we will see what this means exactly and how functions in Scheme can even be passed around and manipulated as first-class items.

For now, we will just explore the basic syntax to define functions in Scheme and how many problems in Scheme can be defined elegantly in a recursive way. First look at the following trivial example of a function definition, then try to implement some of the more complex functions listed below.

```
(define (cube x) (* x x x))
```

Feel free to rephrase this definition by making use of the predefined **expt** function.

4.1 Recursion on numbers

For the following exercices, you might want to look up some of the standard procedures provided by Scheme by clicking here.

Exercise 4

Write a function `minus-pi-over-4` which takes two integer arguments i k ($k >= i > 0$) which computes the sum of the ith to the kth terms of the following series:

$$\text{serie}(n) = \sum_{i=1}^{n} \frac{(-1)^i}{2i-1} = -1 + 1/3 - 1/5 + 1/7 - 1/9 + ... + \frac{(-1)^n}{2n-1}, \text{ si } n > 0$$

For instance, `(minus-pi-over-4 1 4)` should return $-1+1/3-1/5+1/7 = -76/105 = -0.7238095238095239$.
<u>Hint</u>: `minus-pi-over-4`$(i,k) = \frac{(-1)^i}{2i-1} +$ `minus-pi-over-4`$(i+1,k)$

```
(define (minus-pi-over-4 i k)
        ...
        )
```

This auxiliary function may be useful for the next exercise:

```
(define (writeln x)
        (display x)
        (newline))
```

Exercise 5

Write a recursive function `collatz?` which checks the Collatz conjecture for a given integer n. Print all the steps of the check along the way (use `writeln` for instance). The Collatz conjecture states that for any starting number, if we repeatedly apply the f function defined below, we will always eventually reach 1. Use recursion to apply f until reaching 1:

$$f(n) = \begin{cases} n/2 & \text{if } n \text{ is even} \\ 3n+1 & \text{if } n \text{ is odd} \end{cases}$$

Hint: You may find the built-in Scheme functions **`cond`**, `even?` and `odd?` useful.

As a side-exercise, could you define versions of the `even?` and `odd?` predicates yourself?
Hint: Use the built-in Scheme functions `modulo` and `zero?`.

Could you also write `even?` and `odd?` using mutual recursion just for the fun of it?

4.2 Recursion on lists

Using the car and cdr primitives, you can access the head and tail of a list. With this mechanism available, it is easy to recursively scan a list. As always with recursion, you will first need to define a base case: with lists, this is often the case of the empty list `'()`. Then, figure out what should happen when recursing from this base case to the case of a list with one element.

Exercise 6

Write a function `how-many` which takes a first argument x and a second list argument l, and returns the number of times x occurs in l. For example:

```
(how-many 4 '(1 4 5 2 3 4 a b 4 4 c 4))
```

should return 5, because there are 5 occurrences of 4 in this list.

Exercise 7
Write a function `duplicate` which takes a list as input and returns the same list where every element has been duplicated. For example:

```
(duplicate '(Do duck quacks echo?))
```

should return `'(Do Do duck duck quacks quacks echo? echo? )`

Exercise 8
Re-write your `collatz?` function to generate a list of the successive values computed by the process.

```
(collatz-list 13)
-> '(13 40 20 10 5 16 8 4 2 1)
```

5 Special forms and parameter evaluation

From the syntax, it looks like the `if` expression in Scheme is just another function of three arguments: `(if condition true-expr false-expr)`. However, we will see that there is a slight caveat to this approach and that, under the hood, things are a bit different.

Exercise 9
Implement a `(my-if condition true-expr false-expr)` function in terms of **`cond`**. Try it with the following expressions:

```
(define x 4)
(my-if (positive? x) x (- x))
(my-if (positive? x)
       (display "positive")
       (display "negative"))
```

What goes on in your `my-if` function is that the interpreter first evaluates *all* the parameters passed to the the function. In the first example, this does not matter, because `x` and `(- x)` simply evaluate to 4 and -4, without any side-effects. But in the second example, both arguments have the side-effect of displaying some text on the output, and their evaluation produces `#<void>` (this is an implementation choice in racket. The standard simply specifies that `display`, returns "an unspecified value"). In other words, both the true and false blocks are evaluated upon parameter passing, and the function call actually becomes `(my-if #t #<void> #<void>)` but the calls to `display` have already printed out their arguments.

Hygienic macros

Because of this, some constructs in Scheme are actually *special forms* and not functions. In a special form, the arguments are only evaluated when called within the special form, and not when passed as arguments. It is possible to define your own special forms using `define-syntax` and `syntax-rules`. Although you could consult the R^5RS standard for a reference, a simpler tutorial on how Scheme's powerful hygienic macro system works can be found here.

Exercise 10
Rewrite `my-if` using a hygienic macro. Do you observe the problem discovered above?